

Prova Finale di Reti Logiche

Prof. Gianluca Palermo - Anno 2022/2023

Alberto Zheng (Codice Persona 10533714 - Matricola 222422)



POLITECNICO
MILANO 1863

Ingegneria Informatica
Corso di Laurea Triennale - Milano Leonardo

1. Introduzione

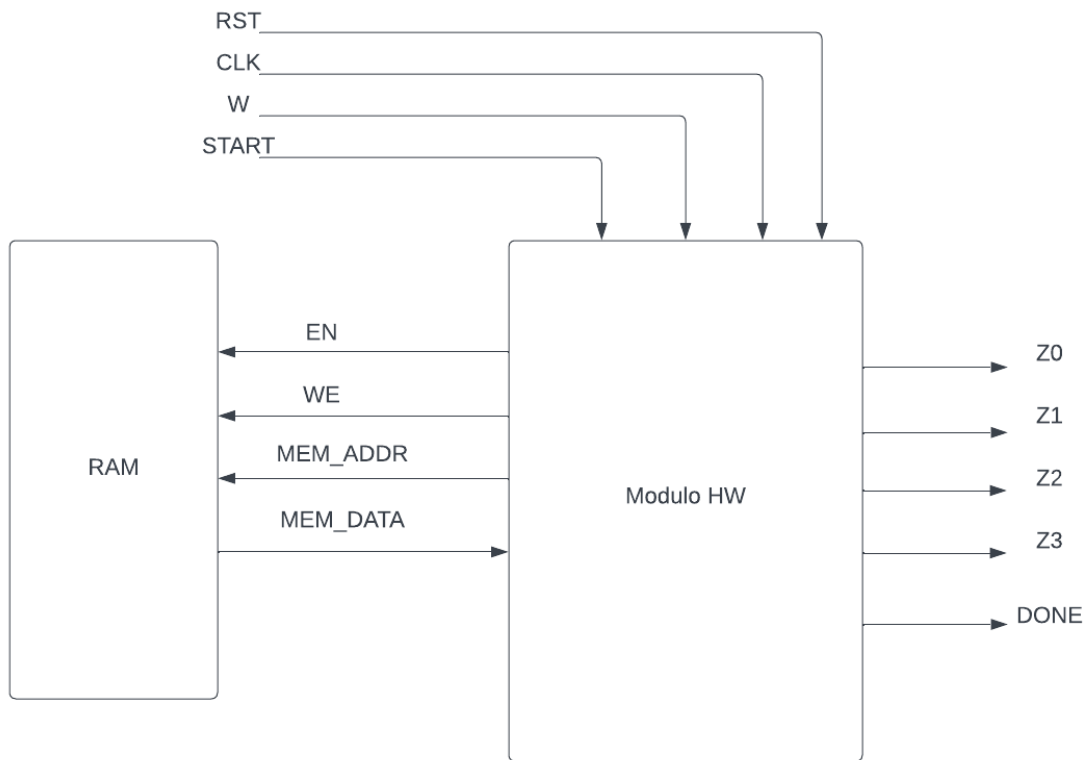
1.1 - Specifiche del progetto

Lo scopo del progetto consiste nell'implementare un modulo HW descritto in VHDL che si interfacci con una memoria RAM tale che riesca ad inoltrare il dato contenuto in tale memoria su un canale di uscita fra i 4 disponibili.

Il modulo è composto da 2 ingressi primari da 1 bit (**W** e **START**) e 5 uscite: quattro da 8 bit (**Z0**, **Z1**, **Z2**, **Z3**) e una da 1 bit (**DONE**).

Ha 2 segnali di ingresso unici per tutto il sistema (**CLK** e **RESET**).

Inoltre, si ha 3 segnali di Output, due da 1 bit **EN** e **WE** e uno da 16 bit **MEM_ADDR** per inviare dati alla memoria e 1 segnale di Input da 8 bit per ricevere dalla memoria (**MEM_DATA**)



Prima di una prima lettura dei dati in ingresso, il modulo si aspetta un segnale di RESET = 1.

Per le letture successive, non ha bisogno di aspettare il segnale di RESET.

Il modulo legge i dati in ingresso con un bit alla volta passati via l'ingresso W sul fronte di salita del clock, quando START = 1

I dati in ingresso possono variare da 2 bit fino a 18 bit.

I primi 2 bit rappresentano su quale canale di uscita indirizzare il dato (00 per Z0, 01 per Z1, 10 per Z2, 11 per Z3).

I restanti 16 indicano l'indirizzo della memoria sul quale deve interfacciarsi per prendere il dato.

Se i bit che indicano l'indirizzo della memoria sono inferiori a 16, allora il modulo estende con dei 0 i restanti bit più significativi.

I dati sulle uscite sono visibili solo quando DONE = 1, per il restante del tempo rimangono a 0. Il segnale di DONE rimane 1 solo per un ciclo di clock.

Il tempo massimo per interfacciarsi con la RAM (dall'istante in cui finisce di leggere i dati in ingresso START passa da 1 a 0 all'istante in cui DONE = 1) è di 20 cicli di clock.

Il segnale EN viene alzato ad 1 quando si deve leggere o scrivere sulla RAM.

Il segnale WE viene alzato ad 1 quando bisogna scrivere sulla RAM (il nostro modulo non ha bisogno di fare nessuna scrittura, quindi questo segnale sarà sempre a 0).

Il segnale MEM_ADDR è il segnale (vettore) di uscita che manda l'indirizzo alla memoria.

Il segnale MEM_DATA che arriva dalla memoria in seguito ad una richiesta di lettura.

1.2 – Interfaccia del componente

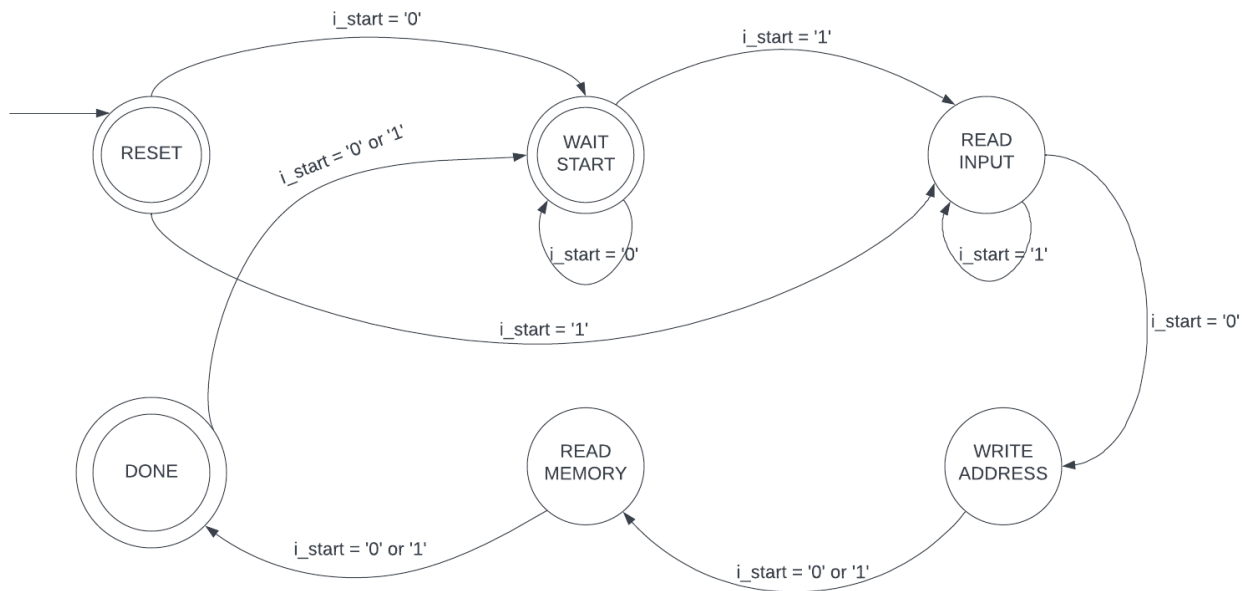
Il modulo è composto dalla seguente interfaccia:

```
entity project_reti_logiche is
  port (
    i_clk    : in std_logic;
    i_rst    : in std_logic;
    i_start  : in std_logic;
    i_w      : in std_logic;

    o_z0     : out std_logic_vector(7 downto 0);
    o_z1     : out std_logic_vector(7 downto 0);
    o_z2     : out std_logic_vector(7 downto 0);
    o_z3     : out std_logic_vector(7 downto 0);
    o_done   : out std_logic;

    o_mem_addr : out std_logic_vector(15 downto 0);
    i_mem_data : in std_logic_vector(7 downto 0);
    o_mem_we   : out std_logic;
    o_mem_en   : out std_logic
  );
end project_reti_logiche;
```

2. Architettura



2.1 – FSM

Nella mia implementazione ho usato una macchina di Moore fatta da 6 stati.

Come si può notare, ci sono stati che si possono accorpare, ma per una maggiore chiarezza del progetto ho deciso di lasciare come in figura.

La macchina è stata progettata in modo che in qualsiasi momento, su qualsiasi stato ci si trovi, quando si riceve in ingresso il segnale $i_rst = '1'$, questa viene resettata su RESET.

I passaggi di stato avvengono sul fronte di salita del clock.

2.2 – Descrizione degli stati

- RESET – la macchina ha ricevuto in ingresso $i_rst = 1$ e ci rimane finchè questo non si abbassi a 0. Una volta abbassato controlla il segnale i_start . Questo stato azzerà tutti i dati salvati sul modulo.
- WAIT_START – Riporta a 0 tutte le uscite e rimane in attesa finchè i_start non si alzi.
- READ_INPUT – legge ed elabora un bit alla volta ricevuto da i_w ad ogni istante di clock finchè i_start rimane alto.
- WRITE_ADDRESS – invia l'indirizzo alla RAM e passa al prossimo stato (ciclo di clock) prima di ricevere il dato dalla memoria.
- READ_MEMORY – legge e salva in un registro interno il dato arrivato dalla RAM dall'indirizzo inviatogli nello stato precedente.
- DONE – alza il segnale o_done ad 1 e mostra sui canali di uscita il dato presente.

2.3 – Registri

Sono stati usati i seguenti segnali:

```
signal current_state : state_type;
```

```
//segnali per salvarmi internamente i dati letti dalla memoria nell'uscita corrispondente
```

```
signal internal_z0 : std_logic_vector(7 downto 0);  
signal internal_z1 : std_logic_vector(7 downto 0);  
signal internal_z2 : std_logic_vector(7 downto 0);  
signal internal_z3 : std_logic_vector(7 downto 0);
```

```
//segnali per salvarmi il dato nell'istante di clock precedente. Usato come segnale ausiliare per  
creare Flip flop invece di latch.
```

```
signal pre_z0 : std_logic_vector(7 downto 0);  
signal pre_z1 : std_logic_vector(7 downto 0);  
signal pre_z2 : std_logic_vector(7 downto 0);  
signal pre_z3 : std_logic_vector(7 downto 0);
```

```
//segnale e suo ausiliare per salvarmi i primi 2 bit letti da i_w indicanti il canale di uscita
```

```
signal channel : std_logic_vector(1 downto 0);  
signal pre_channel : std_logic_vector(1 downto 0);
```

```
//segnale e suo ausiliare per salvarmi l'indirizzo letto da i_w indicanti l'indirizzo.
```

Viene inizializzato tutto a 0 e ad ogni lettura da i_w (dopo i primi 2), vengono shiftati a sinistra i 14 bit meno significanti e aggiunto il nuovo bit come quello il nuovo meno significante

```
signal address : std_logic_vector(15 downto 0);  
signal pre_address : std_logic_vector(15 downto 0);
```

```
//segnale per attivare il processo che alza o_done e mostra sulle uscite i dati corrispondenti
```

```
signal done_reg : std_logic; --Segnale per dire che ho finito di scrivere
```

```
//segnale per contare i numeri di clock da quando i_start = 1. Si blocca a 2 perché serve solo a  
tenere traccia dei primi 2 bit letti per salvarmi il canale, in quanto i restanti bit letti saranno quelli  
che indicano l'indirizzo.
```

Dalla specifica del progetto, alla macchina possono arrivare un massimo di 18 bit e quindi non mi dovevo preoccupare del caso di overflow di bit arrivati da i_w sull'indirizzo.

```
signal clock_counter : integer range 0 to 2;
```

2.4 – Processi

Per l'implementazione si è preferito fare un sistema combinatoria composta da cinque processi. Inizialmente avevo provato a realizzare tutto in un unico processo in quanto il componente era relativamente semplice. Ma poi mi sono reso conto che la manutenzione sarebbe risultata complicata e si formavano diversi latch.

- `lambda: process (i_clk, i_rst, i_start)`
è il processo rappresentante le transizioni della FSM, aggiorna il `current_state` sul fronte di salita del `CLOCK`.
- `delta: process(current_state)`
Controlla i segnali di `o_mem_addr`, `o_mem_en` e `done_reg` durante i vari stati.
- `input: process(i_clk, i_rst, i_start, i_w)`
Controlla i segnali di `channel`, `address` e `clock_counter` sul fronte di salita del clock. Il `channel` e `address` leggono il valore dato da `i_w` quando `i_start = 1`. Altrimenti o sono = 0 o sono = `pre_channel` (`pre_address`) rispettivamente.
Inoltre, questo processo setta su ogni istante di salita del clock i `pre_zX <= internal_zX`.
- `internal_controller: process (i_rst, i_clk)`
Questo processo gestisce i vari segnali `internal_zX`, `pre_channel`, `pre_address` sul fronte di discesa del clock.
I vari segnali `internal_zX` leggono il dato proveniente da `i_mem_data` quando lo stato è `READ_MEMORY`. E' stato scelto di fare la lettura sul fronte di discesa in modo da poter memorizzare il dato all'istante di clock successivo a quello in cui viene passato l'indirizzo alla RAM, così da poter evitare di dover mettere un ciclo di clock extra di idle.
- `output_controller: process (done_reg)`
Controlla gli output `z0`, `z1`, `z2`, `z3` a seconda del segnale `done_reg`.
Quando `done_reg = 1` vengono mostrati sull'uscita i vari valori presenti salvati internamente nei vari `internal_zX`, altrimenti viene mostrato 0 in uscita.

3. Risultati sperimentali

3.1 – Sintesi

Analizzando i reports di sintesi si sono individuate diverse informazioni riguardanti l'implementazione della macchina:

Report Utilization

Nel Report Utilization si trovano le informazioni riguardanti il numero di LUT e registri utilizzati:

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	110	0	134600	0.08
LUT as Logic	110	0	134600	0.08
LUT as Memory	0	0	46200	0.00
Slice Registers	105	0	269200	0.04
Register as Flip Flop	105	0	269200	0.04
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

Si nota che tutti i registri utilizzati risultano essere dei Flip Flop. Non vi è quindi la presenza di Latches che avrebbero potuto creare problemi nei test post sintesi a differenza.

Per eliminare i latch preservando i dati in diversi momenti della FSM si è dovuto utilizzare dei registri duplicati, così facendo sono stati aumentati i numeri dei registri, ma in una quantità accettabile in quanto l'util% dei registri è del solo il 0.04%

Report Timing

Nel Report Timing si trovano le informazioni riguardanti le tempistiche del clock:

Timing Report

```
Slack (MET) :          47.385ns  (required time - arrival time)
  Source:      pre_address_reg[0]/C
                (rising edge-triggered cell FDCE clocked by clock'  {rise@0.000ns fall@50.000ns period=100.000ns})
  Destination: address_reg[0]/D
                (rising edge-triggered cell FDCE clocked by clock  {rise@0.000ns fall@50.000ns period=100.000ns})
  Path Group:   clock
  Path Type:    Setup (Max at Slow Process Corner)
  Requirement:  50.000ns  (clock rise@100.000ns - clock fall@50.000ns)
  Data Path Delay: 1.756ns  (logic 0.791ns (45.046%)  route 0.965ns (54.954%))
  Logic Levels: 1  (LUT6=1)
```

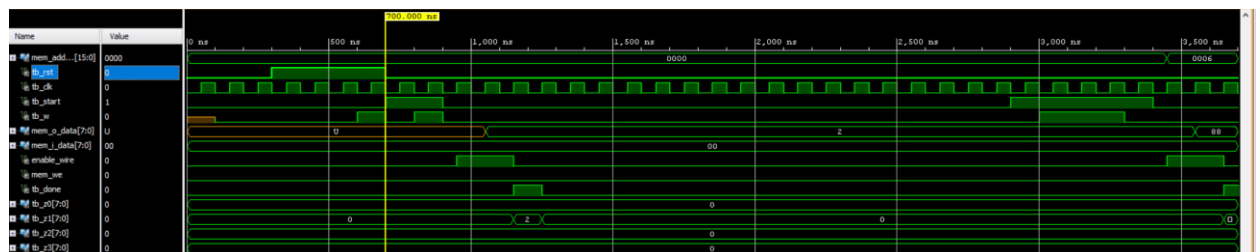
Qui si può notare lo slack pari a 47.385 ns. Quindi per un periodo di clock 100ns, il componente ne utilizza poco più del 50%.

3.2 – Simulazioni

Dopo aver implementato il componente e testato il suo funzionamento, sia in behavioral sia in post-synthesis functional, su tutti i test bench forniti e su altri che ho creato di cui ritenevo utili con esito positivo.

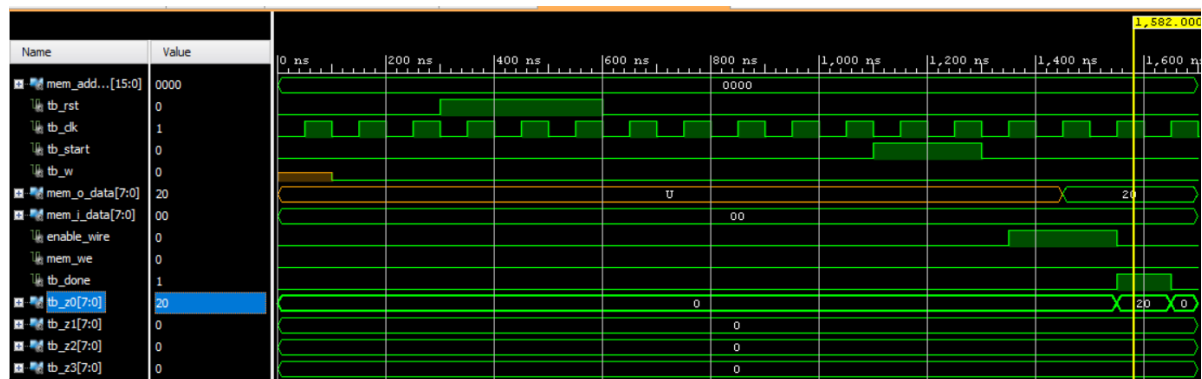
Ho riportato qui i risultati di casi limite in simulazione post sintesi che ritengo rilevanti.

-i_start parte subito al clock successivo ad i_rst



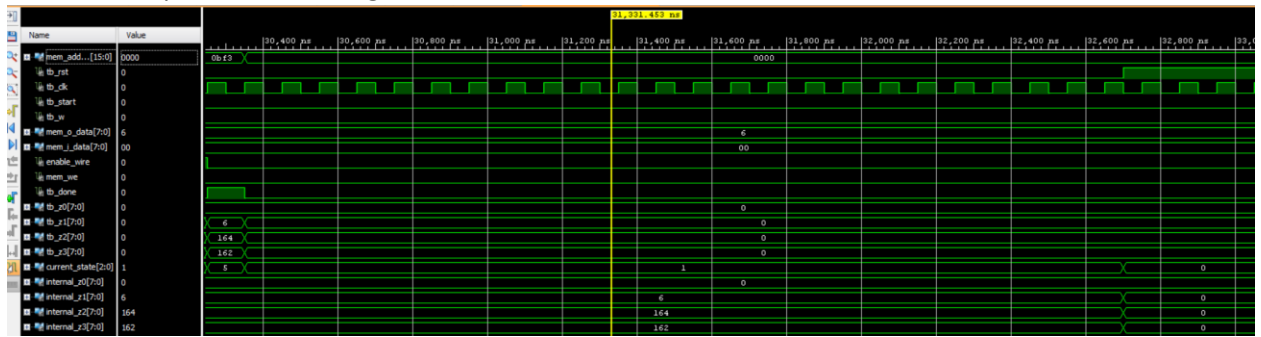
Questo test mostra come si comporta la macchina quando riceve il segnale di i_start subito quando il rst va a 0. Come è possibile vedere dall'istante contrassegnato, la macchina riesce ad ignorare i segnali di i_w alti arrivati negli istanti prima che i_start = 1. Inoltre possiamo notare che non ha bisogno di aspettare un clock extra per cambiare stato da RESET a WAIT_START (questo test mi è stato utile per riuscire a gestire questa situazione).

-0 bit di address da i_w



Qui riusciamo a vedere che il componente riesce a indirizzare correttamente l'indirizzo anche se non riceve nessun bit da i_w dopo i primi 2 indicanti il canale. Inoltre vediamo che il segnale di Done rimane alto per un singolo ciclo di clock.

-comportamento al segnale di reset



In questo test possiamo vedere come il segnale di reset abbia portato a valore iniziale tutti i registri. Da notare come mem_o_data del test bench arrivante dalla RAM (equivalente al segnale di ingresso i_mem_data del componente) non venga influenzato dal reset, e non influisce sulla macchina.

4. Conclusioni

Il componente realizzato ha dimostrato di essere in grado di superare tutti i test eseguiti.

Per completezza, anche se non richiesto dalla specifica, ho simulato e testato tutti i test anche in Post-Implementation Functional con esito positivo.

Si può notare dai test come questa implementazione riesca a leggere dalla RAM e mostrare sui vari output portando il DONE ad 1 in 3 cicli di clock dopo l'istante in cui i_start = 0, rispetto al massimo dei 20 cicli permessi dalla specifica. Considerando il ritardo della RAM di questa specifica, questo numero di cicli clock è il minimo ed ideale.

Inoltre, essendo il componente una macchina combinatoria con i vari processi ben definiti e compiti ben divisi, possiamo dire che questa potrebbe essere una soluzione scalabile e una solida base per implementazioni di specifiche più complesse.